GlowCart — Product Requirements Document (PRD)

Single-Brand 3D Cosmetics E-Commerce Platform | Ghana Market

TABLE OF CONTENTS

---

1. PROJECT OVERVIEW

Project Name: GlowCart **Type:** Single-brand cosmetics e-commerce website (Web + PWA) **Market:** Ghana **Currency:** GHS (Ghanaian Cedi — ₵) **Payment Gateway:** Paystack (supports MTN Mobile Money, Vodafone Cash, AirtelTigo Money, Visa/Mastercard) **Products:** Body sprays, deodorants, skincare (serums, moisturizers, toners, cleansers, sunscreen), lip care, hair care

Vision

A premium, immersive 3D shopping experience tailored for the Ghanaian market where customers can browse, order and track cosmetic products online, while in-store staff use a QR scanner to process walk-in purchases and generate receipts — all managed from a no-code admin dashboard.

Target Users

- **Customers (User):** Ghanaian shoppers aged 16–45 on mobile and desktop
- **Admin:** Brand owner and managers managing inventory, orders and finances
- **Staff/Cashier:** In-store employees using the QR scanner POS

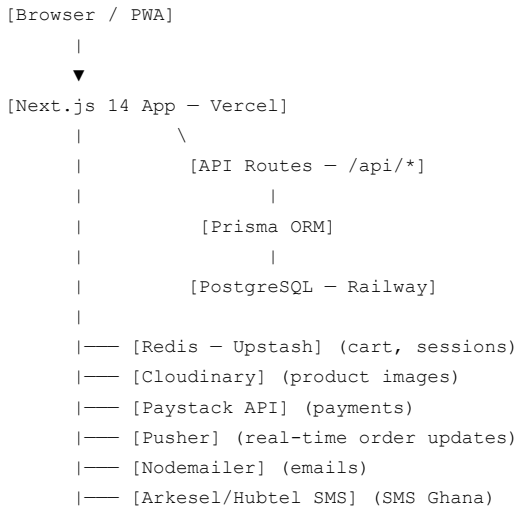
---

2. TECH STACK

Layer	Technology	Reason
Frontend Framework	Next.js 14 (App Router)	SSR, SEO, fast routing
3D & Animation	React Three Fiber, Three.js, Framer Motion	3D product viewer, page transitions
Styling	Tailwind CSS	Utility-first, fast, responsive
State Management	Zustand	Lightweight cart and session state
Backend	Next.js API Routes (Node.js)	Unified codebase, serverless-friendly
Database	PostgreSQL via Prisma ORM	Relational data, type-safe queries
Cache / Session	Redis (Upstash)	Cart state, rate limiting, sessions
Authentication	NextAuth.js	Google, Email/Password, Phone OTP
Payments	Paystack	Ghana Mobile Money + cards
File Storage	Cloudinary	Product images with optimization
QR Generation	qrcode npm package	Auto-generate QR per product
QR/Barcode Scanning	ZXing.js (@zxing/library)	In-browser camera scanning
PDF Receipts	jsPDF + jspdf-autotable	Generate downloadable receipts
Real-time	Pusher (or Socket.IO via Railway)	Live order status updates
Email	Nodemailer + Gmail SMTP	Order confirmation emails
SMS	Hubtel SMS API or Arkesel	Ghana SMS notifications
Analytics Charts	Recharts	Admin dashboard charts
Deployment (Frontend)	Vercel	CDN, edge, free tier available
Deployment (DB + API)	Railway	PostgreSQL hosting + backend
CDN/Security	Cloudflare	HTTPS, DDoS, caching

---

3. SYSTEM ARCHITECTURE



4. USER ROLES & PERMISSIONS

Feature	Customer	Cashier/Staff	Admin
Browse products	✅	✅	✅
Add to cart & checkout	✅	—	—
Track own orders	✅	—	—
QR POS Scanner	—	✅	✅
Generate receipts	—	✅	✅
View all orders	—	✅ (view only)	✅
Update order status	—	—	✅
Add/Edit/Delete products	—	—	✅
View analytics	—	—	✅
Manage users	—	—	✅

PHASE 1 — FOUNDATION

Auth, Product Catalog, Cart & Checkout

Objective

Build the complete customer-facing shopping experience: user registration/login, browsing products by category, adding to cart, and completing payment via Paystack (Mobile Money + cards). Orders are saved to the database.

Features

Authentication

- Register with email + password
- Login with email + password
- Google OAuth login
- Phone number + OTP login (via Arkesel or Africa's Talking)
- Password reset via email
- Protected routes (redirect to login if unauthenticated)
- JWT session via NextAuth.js
- User profile page (edit name, phone, address)

Product Catalog

- Homepage with featured products section
- All products page with filters:
  - Category filter (Body Spray, Skincare, Deodorant, Lip Care, Hair Care)
  - Price range slider (₵0 – ₵500+)
  - Sort by: Newest, Price Low–High, Price High–Low, Most Popular
- Product detail page:
  - Multiple product images (carousel)
  - Product name, price in GHS, description, ingredients
  - Quantity selector
  - "Add to Cart" button
  - Related products section
- Search bar with real-time results (debounced, 300ms)
- Product badges: "New", "Best Seller", "Low Stock"

Shopping Cart

- Persistent cart (saved to DB for logged-in users, localStorage for guests)
- Add / remove / update quantity in cart
- Cart slide-out drawer (visible from all pages)
- Cart total in GHS with itemized list
- "Proceed to Checkout" button

Checkout

- Shipping address form (name, address, city, region, phone)
- Ghana regions dropdown (Greater Accra, Ashanti, Western, etc.)
- Order summary (items, subtotal, delivery fee, total)
- Delivery fee logic: Free above ₵200, flat ₵15 below
- Pay with Paystack button (Mobile Money or Card)
- Paystack popup integration (client-side)
- On payment success: create Order in DB, clear cart, redirect to /order-confirmed
- On payment failure: show error toast, stay on checkout
- Order confirmation page with order number and summary

Folder Structure

```
/app
  /page.tsx                  ← Homepage
  /products
    /page.tsx                ← All products
    /[slug]/page.tsx         ← Product detail
  /cart/page.tsx
  /checkout/page.tsx
```

```
  /order-confirmed/page.tsx
  /auth
    /login/page.tsx
    /register/page.tsx
    /forgot-password/page.tsx
  /profile/page.tsx

/components
  /ui                        ← Buttons, Input, Badge, Modal, Toast
  /layout                    ← Navbar, Footer, CartDrawer
  /product                   ← ProductCard, ProductGrid, ProductCarousel
  /checkout                  ← CheckoutForm, PaystackButton, OrderSummary

/lib
  /prisma.ts                 ← Prisma client singleton
  /paystack.ts               ← Paystack helper functions
  /auth.ts                   ← NextAuth config
  /utils.ts                  ← formatGHS(), slugify(), etc.

/api
  /auth/[...nextauth]/route.ts
  /products/route.ts          ← GET all products (with filters)
  /products/[id]/route.ts     ← GET single product
  /cart/route.ts              ← GET, POST, PUT, DELETE cart items
  /orders/route.ts            ← POST create order
  /orders/[id]/route.ts       ← GET order detail
  /paystack/verify/route.ts   ← POST verify payment after callback

/prisma
  /schema.prisma
```

Database Tables (Phase 1)

```
model User {
  id            String    @id @default(cuid())
  name          String
  email         String    @unique
  phone         String?
  passwordHash  String?
  role          Role      @default(CUSTOMER)
  address       String?
  city          String?
  region        String?
  createdAt     DateTime  @default(now())
  orders        Order[]
  cartItems     CartItem[]
}

model Product {
  id            String    @id @default(cuid())
  name          String
  slug          String    @unique
  description   String
  price         Float
  category      Category
  images        String[]  // Cloudinary URLs
  stock         Int       @default(0)
  isFeatured    Boolean   @default(false)
  isPublished   Boolean   @default(true)
  badge         String?   // "New", "Best Seller", "Low Stock"
```

```
  qrCode        String?   // Base64 QR image or URL
  createdAt     DateTime  @default(now())
  orderItems    OrderItem[]
  cartItems     CartItem[]
}

model CartItem {
  id         String  @id @default(cuid())
  userId     String
  productId  String
  quantity   Int
  user       User    @relation(fields: [userId], references: [id])
  product    Product @relation(fields: [productId], references: [id])
}

model Order {
  id              String      @id @default(cuid())
  userId          String
  user            User        @relation(fields: [userId], references: [id])
  items           OrderItem[]
  subtotal        Float
  deliveryFee     Float
  total           Float
  status          OrderStatus @default(PLACED)
  paymentRef      String?     // Paystack reference
  paymentStatus   String      @default("pending")
  shippingAddress String
  city            String
  region          String
  phone           String
  trackingCode    String      @unique @default(cuid())
  createdAt       DateTime    @default(now())
  updatedAt       DateTime    @updatedAt
}

model OrderItem {
  id         String  @id @default(cuid())
  orderId    String
  productId  String
  quantity   Int
  unitPrice  Float
  order      Order   @relation(fields: [orderId], references: [id])
  product    Product @relation(fields: [productId], references: [id])
}

enum Role {
  CUSTOMER
  CASHIER
  ADMIN
}

enum Category {
  BODY_SPRAY
  SKINCARE
  DEODORANT
  LIP_CARE
  HAIR_CARE
  OTHER
}

enum OrderStatus {
  PLACED
  CONFIRMED
```

```
  PACKED
  SHIPPED
  DELIVERED
  CANCELLED
}
```

Key API Endpoints (Phase 1)

Method	Endpoint	Description
GET	/api/products	Get all products (with ?category=, ?sort=, ?search=)
GET	/api/products/[id]	Get single product by ID or slug
GET	/api/cart	Get cart for current user
POST	/api/cart	Add item to cart
PUT	/api/cart/[itemId]	Update cart item quantity
DELETE	/api/cart/[itemId]	Remove item from cart
POST	/api/orders	Create new order after payment
GET	/api/orders/[id]	Get order by ID
POST	/api/paystack/verify	Verify Paystack payment reference

AI Prompt for Phase 1

> You are a senior full-stack developer. Build Phase 1 of a cosmetics e-commerce web app called **GlowCart** for the Ghanaian market using **Next.js 14 (App Router), PostgreSQL with Prisma ORM, NextAuth.js, Zustand, Tailwind CSS, and Paystack** for payments.
>
> **What to build:**
>
> 1. Set up the Next.js 14 project with Tailwind CSS, Prisma, NextAuth.js (Email/Password + Google OAuth), and Zustand for cart state.
> 2. Create a Prisma schema with these models: User (id, name, email, phone, passwordHash, role [CUSTOMER/CASHIER/ADMIN], address, city, region), Product (id, name, slug, description, price [Float], category [enum: BODY_SPRAY, SKINCARE, DEODORANT, LIP_CARE, HAIR_CARE], images [String array], stock, isFeatured, isPublished, badge, qrCode, createdAt), CartItem (userId, productId, quantity), Order (id, userId, items, subtotal, deliveryFee, total, status [enum: PLACED/CONFIRMED/PACKED/SHIPPED/DELIVERED/CANCELLED], paymentRef, shippingAddress, city, region, phone, trackingCode), OrderItem (orderId, productId, quantity, unitPrice).
> 3. Build these pages: Homepage (with featured products grid), /products (all products with category filter, price range filter, sort dropdown, and search bar), /products/[slug] (product detail with image carousel, quantity selector, add-to-cart), /cart (cart page with quantities, totals, remove items), /checkout (address form with Ghana regions dropdown, Paystack integration, order summary), /order-confirmed (success page with order details), /auth/login, /auth/register, /auth/forgot-password, /profile.
> 4. All prices must display in GHS (₵). Delivery fee is ₵15 flat rate, free above ₵200 subtotal.
> 5. Integrate Paystack using their JavaScript popup (not redirect). On successful payment, call POST /api/paystack/verify with the payment reference to confirm the transaction, then create the order in the database and clear the cart.
> 6. Cart must persist to PostgreSQL for logged-in users and to localStorage for guests. Merge guest cart into DB cart on

login.

7. Build REST API routes under /api: products (GET with filters), cart (GET/POST/PUT/DELETE), orders (POST create, GET by ID), paystack/verify (POST).
8. Use Cloudinary URLs for product images. Store them as a String array in the Product model.
9. Style everything with Tailwind CSS. Use a luxury cosmetics color palette: deep rose (#C2185B), soft cream (#FFF8F0), and charcoal (#1C1C1C). Mobile-first, fully responsive.
10. Add toast notifications (react-hot-toast) for: item added to cart, order placed, payment errors.

Include all environment variable names needed in a .env.example file.

PHASE 2 — ADMIN PANEL

Product CRUD, Order Management & Analytics Dashboard

Objective

Build a complete no-code admin panel where the brand owner can add/edit/delete products (with image upload), manage all orders (update status, view details), view customer list, and see an analytics dashboard with daily/weekly/monthly revenue charts and top-selling products — all without touching code.

Features

Admin Authentication & Access Control

- Admin login at /admin/login (separate from customer login)
- Role check middleware: only users with role=ADMIN can access /admin/*
- Auto-redirect to /admin/login if unauthorized

Dashboard / Analytics Page (/admin)

- Total revenue today, this week, this month (cards at top)
- Line chart: Daily revenue for last 30 days (using Recharts)
- Bar chart: Revenue by product category
- Pie chart: Order status breakdown (Placed / Confirmed / Packed / Shipped / Delivered)
- Top 5 best-selling products (with image, name, units sold, revenue)
- Recent 10 orders live feed
- Total customers count, new customers today
- Low stock alert: products with stock ≤ 5

Product Management (/admin/products)

- Table view: all products with columns (image thumbnail, name, category, price, stock, status, actions)
- Pagination (10 per page)
- Search products by name

- Filter by category and published/draft status
- Add Product button → modal or full page form with:
  - Product name (text)
  - Slug (auto-generated from name, editable)
  - Description (rich text — React Quill)
  - Price (GHS number input)
  - Category (dropdown)
  - Stock quantity (number)
  - Badge (optional: New / Best Seller / Low Stock / empty)
  - Images: drag-and-drop multi-image upload → Cloudinary (up to 5 images)
  - Published toggle (draft/live)
  - Featured toggle (show on homepage)
  - On save: auto-generate QR code for this product (encode product ID) and store in DB
- Edit Product: same form pre-filled
- Delete Product: confirm dialog, soft delete (set isPublished=false) or hard delete
- View Product QR code: show QR code image, download button

Order Management (/admin/orders)

- Table: all orders with columns (order ID, customer name, total ₵, status badge, date, actions)
- Filter by: status, date range, region
- Click order → detail view:
  - Customer info, shipping address
  - Ordered items with images
  - Payment reference
  - Current status with color badge
  - Status update dropdown (Confirmed → Packed → Shipped → Delivered → Cancelled)
  - On status change: auto-send SMS/email to customer
- Export orders to CSV

Customer Management (/admin/customers)

- Table: all customers (name, email, phone, region, total orders, total spent, joined date)
- Click customer → view their order history
- Block/suspend account toggle

Folder Structure

```
/app/admin
  /page.tsx                  ← Dashboard / Analytics
  /products/page.tsx         ← Product list
```

```
  /products/new/page.tsx      ← Add product form
  /products/[id]/edit/page.tsx ← Edit product form
  /orders/page.tsx            ← Order list
  /orders/[id]/page.tsx       ← Order detail
  /customers/page.tsx         ← Customer list
  /login/page.tsx             ← Admin login

/components/admin
  /AdminLayout.tsx            ← Sidebar + header layout
  /Sidebar.tsx
  /StatsCard.tsx
  /RevenueChart.tsx
  /CategoryChart.tsx
  /OrderStatusChart.tsx
  /TopProductsTable.tsx
  /ProductForm.tsx
  /ProductTable.tsx
  /OrderTable.tsx
  /OrderDetail.tsx
  /ImageUploader.tsx          ← Drag-and-drop Cloudinary upload
  /QRCodeViewer.tsx

/api/admin
  /stats/route.ts             ← GET analytics data
  /products/route.ts          ← GET all, POST create
  /products/[id]/route.ts     ← GET, PUT, DELETE
  /products/[id]/qr/route.ts  ← GET QR code for product
  /orders/route.ts            ← GET all orders
  /orders/[id]/route.ts       ← GET, PATCH status
  /customers/route.ts         ← GET all customers
  /upload/route.ts            ← POST image to Cloudinary
```

Key API Endpoints (Phase 2)

Method	Endpoint	Description
GET	/api/admin/stats	Revenue totals, chart data, top products
GET	/api/admin/products	All products (paginated, filtered)
POST	/api/admin/products	Create product + auto-generate QR
PUT	/api/admin/products/[id]	Update product
DELETE	/api/admin/products/[id]	Delete product
GET	/api/admin/orders	All orders (filtered, paginated)
PATCH	/api/admin/orders/[id]	Update order status
GET	/api/admin/customers	All customers
POST	/api/admin/upload	Upload image to Cloudinary

Analytics Data Shape (from /api/admin/stats)

```
{
  "revenueToday": 850.00,
```

```
  "revenueThisWeek": 4200.00,
  "revenueThisMonth": 16500.00,
  "totalOrders": 312,
  "totalCustomers": 189,
  "newCustomersToday": 4,
  "dailyRevenue": [
    { "date": "2025-06-01", "revenue": 850.00 },
    { "date": "2025-05-31", "revenue": 1100.00 }
  ],
  "revenueByCategory": [
    { "category": "SKINCARE", "revenue": 7800.00 },
    { "category": "BODY_SPRAY", "revenue": 5200.00 }
  ],
  "orderStatusBreakdown": [
    { "status": "DELIVERED", "count": 200 },
    { "status": "PLACED", "count": 45 }
  ],
  "topProducts": [
    { "id": "...", "name": "Glow Serum", "image": "...", "unitsSold": 87, "revenue": 3480.00 }
  ],
  "lowStockProducts": [
    { "id": "...", "name": "Rose Body Spray", "stock": 3 }
  ]
}
```

AI Prompt for Phase 2

> You are a senior full-stack developer. Build Phase 2 of **GlowCart**, a cosmetics e-commerce app in Next.js 14. Phase 1 is complete (products, cart, checkout, Paystack payments, Prisma + PostgreSQL). Now build the **Admin Panel**.
>
> **Important context:** The app uses Next.js 14 App Router, Prisma ORM with PostgreSQL, NextAuth.js (users have a `role` field: CUSTOMER, CASHIER, ADMIN), Tailwind CSS, and Cloudinary for images. All prices are in GHS (Ghanaian Cedi ₵).
>
> **What to build:**
>
> 1. Create an admin middleware: any route under /admin/* must check the session user role equals ADMIN. If not, redirect to /admin/login.
> 2. Build an admin layout (/app/admin/layout.tsx) with a collapsible sidebar containing links to: Dashboard, Products, Orders, Customers. Sidebar should show the brand logo and admin user's name.
> 3. **Dashboard page (/admin):** Show 4 stat cards at top (Revenue Today, Revenue This Week, Revenue This Month, Total Orders). Below that: a Recharts LineChart of daily revenue for last 30 days, a BarChart of revenue by product category, a PieChart of order status distribution. Below charts: a "Top 5 Products" table and a "Low Stock Alerts" list (products with stock ≤ 5). Fetch all data from GET /api/admin/stats.
> 4. **Products page (/admin/products):** Paginated table (10/page) showing all products with thumbnail, name, category, price (₵), stock, published badge, and Edit/Delete action buttons. Include search input and category filter dropdown. "Add Product" button routes to /admin/products/new.
> 5. **Add/Edit Product form:** Fields: name (auto-generates slug), description (use React Quill rich text editor), price (number, GHS), category (dropdown), stock (number), badge (optional dropdown: New/Best Seller/Low Stock), featured toggle, published toggle, and a drag-and-drop image uploader that uploads to Cloudinary via POST /api/admin/upload and stores up to 5 image URLs. On save, auto-generate a QR code using the `qrcode` npm package encoding the product's ID, store the QR as a base64 string in the product record.
> 6. **Orders page (/admin/orders):** Table showing order ID (truncated), customer name, total ₵, status badge (color-coded), date, and a "View" button. Filter by status and date range. Clicking a row opens the order detail page showing: customer info, shipping address, list of ordered items with images, payment reference, and a status update dropdown (PLACED → CONFIRMED → PACKED → SHIPPED → DELIVERED / CANCELLED). When status changes, call PATCH

/api/admin/orders/[id] and (for now) log that a notification would be sent.

7. **Customers page (/admin/customers):** Table of all users with CUSTOMER role showing name, email, phone, region, total orders count, total amount spent (₵), and join date. Clicking a row shows their order history in a side panel.
8. Build all required API routes under /api/admin/ — protect each with role check. The /api/admin/stats route should query the database using Prisma aggregations for revenue sums grouped by day, category breakdowns, and order status counts.
9. Style with Tailwind CSS. Admin theme: white background, slate-800 sidebar, rose-600 accent. Fully responsive (sidebar collapses to hamburger on mobile).
10. Add a CSV export button on the Orders page that downloads order data as a CSV file using the `papaparse` library.

PHASE 3 — 3D VISUAL EXPERIENCE & DESIGN

Immersive UI Without 3D Models

Objective

Create a visually stunning, luxury cosmetics website feel using CSS 3D transforms, React Three Fiber abstract scenes, Framer Motion animations, and Tailwind CSS — without needing actual 3D product model files (.glb). Every interaction should feel premium.

Features & Implementation

Homepage 3D Hero Section (CSS + Three.js)

- Full-width hero with animated floating orbs/spheres in brand colors using React Three Fiber
- Spheres use MeshStandardMaterial with rose/gold/cream colors and subtle metallic sheen
- Particles (Three.js Points) float and drift slowly in the background
- Hero text animates in with Framer Motion (staggered fade + slide up)
- CTA button with shimmer effect on hover
- NO actual product 3D models needed — the visual is abstract and branded

Product Card 3D Tilt Effect (CSS Transform)

- Each product card responds to mouse movement with CSS perspective tilt
- On hover: card tilts 8–12 degrees based on cursor position (vanilla-tilt or custom JS)
- Soft drop shadow intensifies on hover
- Quick-add button slides up from bottom on hover

Product Detail — 360° CSS Spinner (Workaround for no 3D model)

- Instead of a 3D model, use multiple product photos in a sequence
- If 4+ images uploaded, enable a drag/swipe 360° illusion using CSS transitions
- A “Spin” button cycles through all images on a loop at 150ms intervals

- If fewer than 4 images, show standard carousel with zoom-on-click

Page Transitions (Framer Motion)

- All page transitions use a slide-up curtain animation (rose-colored div sweeps across screen between routes)
- Shared layout transitions: product card expands into product detail page

Scroll Animations

- Product grid: cards fade and slide in as user scrolls (Framer Motion viewport detection)
- Stats/numbers on homepage animate counting up when they enter viewport
- Category section: horizontal scroll with snap (mobile-friendly)

Loading States

- Custom luxury loading spinner (animated rose petals or rotating ring in brand color)
- Skeleton loaders for product cards (pulse animation)
- Image lazy loading with blur-to-sharp transition (Next.js Image blur placeholder)

Floating Cart Button

- Sticky floating cart icon (bottom-right on mobile)
- Badge with item count, bounces on item add
- Cart drawer slides in from right with Framer Motion spring animation

---

Three.js Scene Details (Hero Section)

```
// Scene setup — no 3D models needed
// Use Three.js primitive geometries styled as abstract cosmetics-inspired shapes

const shapes = [
  { geometry: SphereGeometry(0.8), position: [-2, 1, 0], color: '#C2185B' },  // Rose sphere
  { geometry: SphereGeometry(0.5), position: [2, -1, 1], color: '#F8BBD0' },  // Pink sphere
  { geometry: TorusGeometry(0.6, 0.2), position: [0, 2, -1], color: '#FFD700' }, // Gold ring
  { geometry: CylinderGeometry(0.3, 0.3, 1.5), position: [3, 0, 0], color: '#FFF8F0' }, // Cream cylinder (bottle
]

// Animate: slow rotation on all shapes + gentle floating (sine wave y-axis)
// Post-processing: add bloom effect for glow feel (drei: EffectComposer > Bloom)
// Lighting: ambient rose light + directional white light
```

Color & Typography System

```
/* Brand Colors */
--rose-primary: #C2185B
--rose-light: #F8BBD0
--gold-accent: #D4AF37
--cream: #FFF8F0
--charcoal: #1C1C1C
--slate: #4A4A4A

/* Typography */
--font-display: 'Cormorant Garamond' (Google Fonts) — headings, luxury feel
--font-body: 'DM Sans' (Google Fonts) — body text, clean and modern
```

AI Prompt for Phase 3

> You are a senior frontend developer and UI/UX designer. Build Phase 3 of **GlowCart**, a luxury cosmetics e-commerce site in Next.js 14 + Tailwind CSS + Framer Motion + React Three Fiber. Phase 1 (shopping flow) and Phase 2 (admin) are already complete.
>
> **Important:** There are NO 3D product model files (.glb/.gltf). All 3D visuals must be created using Three.js primitive geometries and CSS transforms. Do not import or reference any .glb files.
>
> **What to build:**
>
> 1. **Hero Section (React Three Fiber):** Create a full-width hero with a Three.js canvas behind the headline text. The scene contains: 3 slowly rotating spheres (MeshStandardMaterial, colors: #C2185B, #F8BBD0, #D4AF37), 1 torus ring (gold, slow rotation), 1 cylinder (cream color, representing an abstract bottle shape), and a particle system (500 small dots drifting slowly). Add a Bloom post-processing effect using @react-three/postprocessing. Lighting: PointLight at top-left (rose, intensity 2) + AmbientLight (white, intensity 0.5). Canvas is interactive (OrbitControls disabled, but objects subtly follow mouse position). The hero text "Glow. Nourish. Shine." animates in with Framer Motion staggered delays.
> 2. **Product Card Tilt Effect:** Apply CSS perspective 3D tilt to every product card using a custom React hook `useTilt` that tracks mousemove and applies `transform: perspective(800px) rotateX(Xdeg) rotateY(Ydeg)` based on cursor position relative to card center. Max tilt: 12 degrees. Add a subtle shine overlay div that moves with cursor to simulate light reflection.
> 3. **360° Image Spinner on Product Detail Page:** On the product detail page, if the product has 3 or more images, show a "360° View" toggle button. When activated, cycle through all product images at 200ms intervals giving the illusion of rotation. Allow click-drag to manually scrub through images. Show a circular progress indicator while cycling.
> 4. **Page Transitions:** Wrap the app in a Framer Motion AnimatePresence layout. Between route changes, animate a rose (#C2185B) full-screen curtain that sweeps up and down. Use `usePathname()` from next/navigation as the key.
> 5. **Scroll Animations:** Use Framer Motion's `whileInView` to animate product cards: each card fades in and slides up 30px as it enters the viewport. Add `staggerChildren: 0.08` for grid stagger. Category section on homepage should be a horizontal scroll container with `scroll-snap-type: x mandatory` and CSS snap points.
> 6. **Typography:** Import and apply 'Cormorant Garamond' (from Google Fonts) for all headings (h1–h3) and 'DM Sans' for all body text. Configure both in Tailwind's `fontFamily` config.
> 7. **Skeleton Loaders:** Build a `ProductCardSkeleton` component that matches the product card dimensions with animated pulse (Tailwind animate-pulse). Show 8 skeletons while products are loading.
> 8. **Floating Cart Button:** On mobile (md: hidden), show a fixed bottom-right button with the cart icon and item count badge. Animate the badge with a spring scale bounce when item count increases (Framer Motion).
> 9. **Loading Screen:** Create a custom full-screen loading animation for initial app load — an animated rose (#C2185B) ring that draws itself (SVG stroke-dashoffset animation) with the brand name fading in below it. Show for 1.5 seconds then fade out.
> 10. Use Tailwind CSS for all other styling. All animations must be smooth at 60fps and must not cause layout shift (use transform/opacity only, never width/height animations).

PHASE 4 — QR / BARCODE POS SCANNER & RECEIPT GENERATOR

In-Store Staff Tool

Objective

Build a dedicated in-store POS (Point of Sale) tool accessible to CASHIER and ADMIN roles. Staff open the scanner on a tablet or phone, scan the QR code on each product, see the item added to a running session list with real-time total in GHS, adjust quantities if needed, then generate a printable/downloadable PDF receipt. This is entirely browser-based — no app installation required.

Features

Access Control

- Scanner page at /pos (protected: CASHIER + ADMIN only)
- Separate /pos/login page for staff sign-in
- Cashier sees only their own scan sessions; Admin sees all sessions

QR Scanning Interface

- Button to "Start Camera" — requests camera permissions in browser
- Uses @zxing/library (ZXing.js) to decode QR codes from the live camera feed
- Scanning is continuous (no need to press a button per scan)
- On successful scan:
  - Decode product ID from QR payload
  - Call GET /api/products/[id] to fetch product details
  - Show a success flash + beep sound (Web Audio API)
  - Add item to the scan session list
  - If item already in list, increment quantity by 1
- On unrecognized QR code: show "Unknown product" error toast
- Manual product search: if QR fails, staff can search by product name and add manually

Scan Session List

- Scrollable list showing all scanned items:
  - Product image thumbnail
  - Product name
  - Unit price (₵)
  - Quantity (editable spinner)
  - Line total (₵)
  - Remove button
- Running subtotal, tax (0% — Ghana cosmetics exempt from VAT for now, but configurable), and grand total all update in real time
- "Clear All" button with confirmation dialog

Receipt Generation

- "Generate Receipt" button triggers jsPDF to create a PDF
- Receipt layout:

- Brand logo at top
- Store name, address, phone, date, time
- Cashier name
- Receipt number (auto-incremented, e.g., REC-2025-0087)
- Itemized table: item name, qty, unit price, line total
- Subtotal, discount (if any), total
- Payment method selector (Cash / Mobile Money / Card) before generating
- "Thank you" message + return policy at bottom

- Actions after generating:
  - Download PDF button
  - Print button (window.print() with print-specific CSS)
  - "New Sale" button to clear session and start fresh

Session History (/pos/history)

- Table of all past scan sessions by this cashier
- Columns: Receipt #, date/time, items count, total ₵, payment method, cashier
- Click row to view/reprint receipt

QR Code Format

Each product QR code encodes a signed JSON payload:

```
{
  "pid": "clx9k2abc...",
  "store": "GlowCart",
  "v": 1
}
```

The payload is Base64-encoded so it's compact. On scan, decode Base64 → parse JSON → extract `pid` → query product.

Database Tables (Phase 4)

```
model ScanSession {
  id            String      @id @default(cuid())
  cashierId     String
  cashier       User        @relation(fields: [cashierId], references: [id])
  items         ScanItem[]
  subtotal      Float
  total         Float
  paymentMethod String      // "CASH", "MOBILE_MONEY", "CARD"
  receiptNumber String      @unique
  receiptUrl    String?
  createdAt     DateTime    @default(now())
}

model ScanItem {
  id          String      @id @default(cuid())
  sessionId   String
```

```
  session     ScanSession @relation(fields: [sessionId], references: [id])
  productId   String
  product     Product     @relation(fields: [productId], references: [id])
  quantity    Int
  unitPrice   Float
  lineTotal   Float
}
```

Key API Endpoints (Phase 4)

Method	Endpoint	Description
GET	/api/products/[id]	Fetch product by ID (for QR scan lookup)
POST	/api/pos/sessions	Save completed scan session
GET	/api/pos/sessions	Get all sessions for current cashier
GET	/api/pos/sessions/[id]	Get session detail
GET	/api/pos/receipt/[id]	Get receipt data for re-print

AI Prompt for Phase 4

> You are a senior full-stack developer. Build Phase 4 of **GlowCart** — the **in-store QR/Barcode POS Scanner and Receipt Generator** for staff.
>
> **Context:** The app is Next.js 14, Prisma + PostgreSQL, NextAuth.js (roles: CUSTOMER, CASHIER, ADMIN), Tailwind CSS. Products already exist in the DB and each has a `qrCode` field storing a Base64 QR image (generated in Phase 2). The QR code encodes this Base64-encoded JSON: `{ "pid": "<productId>", "store": "GlowCart", "v": 1 }`.
>
> **What to build:**
>
> 1. Create a /pos route group protected by middleware — only CASHIER and ADMIN roles can access. Unauthenticated users redirect to /pos/login.
> 2. **Main Scanner Page (/pos):** A mobile-first layout (designed for tablets/phones). Top section: a live camera preview div (`<video>` element). Below it: the scanned items list. Bottom: total and action buttons. Use `@zxing/library` (npm: @zxing/browser and @zxing/library) to continuously decode QR codes from the camera. On successful decode: Base64-decode the payload, parse JSON, extract `pid`, call GET /api/products/[pid] to get product details, then add to the session items state. Play a short beep using the Web Audio API on each successful scan. If the same product is scanned again, increment its quantity.
> 3. **Session Items List:** Each item shows a small image thumbnail, product name, quantity (with + and - buttons), unit price in ₵, and line total (₵). A "×" button removes the item. Below the list show: Subtotal (₵), and Grand Total (₵) updating in real-time as items change.
> 4. **Manual Search Fallback:** A text input above the camera feed where staff can type a product name and see live suggestions (fetched from /api/products?search=...). Clicking a suggestion adds it to the session.
> 5. **Generate Receipt Flow:** A "Generate Receipt" button opens a modal asking for payment method (Cash / Mobile Money / Card). On confirm: use `jsPDF` and `jspdf-autotable` to generate a PDF receipt. The receipt must include: GlowCart logo (inline base64 PNG), store name "GlowCart", date/time, cashier name (from session), a unique receipt number (format: REC-YYYYMMDD-XXXX where XXXX is 4-digit sequential), itemized table with columns (Product Name | Qty | Unit Price ₵ | Total ₵), subtotal row, grand total row, payment method, and a thank-you message. Auto-save the session to the database via POST /api/pos/sessions. Offer a "Download PDF" button and a "Print" button.

6. **Session History (/pos/history):** A table showing all past sessions for the logged-in cashier. Columns: Receipt #, Date, Time, Items Count, Total ₵, Payment Method, Reprint button. Fetched from GET /api/pos/sessions.
7. Add the ScanSession and ScanItem models to Prisma schema (as defined in the PRD). ScanSession has: cashierId, items, subtotal, total, paymentMethod, receiptNumber (unique, auto-incremented), createdAt. ScanItem has: sessionId, productId, quantity, unitPrice, lineTotal.
8. Build API routes: GET /api/products/[id] (already exists — ensure it works), POST /api/pos/sessions (save session + items), GET /api/pos/sessions (list sessions for current user), GET /api/pos/sessions/[id] (session detail with items).
9. The UI should be designed for a 10-inch tablet in landscape mode primarily but also work on a phone portrait. Use large tap targets (min 44px), high contrast, and easy-to-read font size (16px minimum). Use Tailwind CSS.
10. Add a "New Sale" button that clears all session state (with a confirm dialog) so the cashier can start a fresh scan session.

PHASE 5 — REAL-TIME ORDER TRACKING & NOTIFICATIONS

Live Updates for Customers + Automated SMS/Email

Objective

Customers can track their order status in real time on a tracking page. When admin updates an order status, the change reflects instantly on the customer's screen AND triggers an SMS (via Arkesel) and email (via Nodemailer/Gmail SMTP) notification to the customer.

Features

Order Tracking Page (/track or /orders/[trackingCode])

- Publicly accessible (no login required if customer has tracking code)
- Also accessible from user dashboard under order history
- Visual progress timeline: 5 steps: Order Placed → Confirmed → Packed → Shipped → Delivered
- Current step highlighted in rose (#C2185B), completed steps in green, future steps in gray
- Real-time update: page subscribes to Pusher channel `order-<trackingCode>`
- When admin updates status, Pusher broadcasts event → customer's page updates timeline instantly without refresh
- Order details below timeline: items, total, delivery address, estimated delivery note

User Order History (/dashboard/orders)

- List of all past and current orders
- Each order card shows: order ID (truncated), date, total, status badge, "Track Order" button
- Filter by status

Admin Notifications Trigger

- When admin changes order status on /admin/orders/[id]:
  - Call Pusher server-side to broadcast `status-updated` event to channel `order-<trackingCode>`
  - Call Arkesel API (or Africa's Talking) to send SMS to customer's phone number

- Call Nodemailer to send email to customer's email address
- SMS and email templates for each status change

SMS Templates (Ghana — Arkesel)

```
PLACED:    "Hi [name], your GlowCart order #[id] has been placed! ₵[total]. Track: glowcart.com/track/[code]"
CONFIRMED: "Hi [name], your order #[id] is confirmed and being prepared!"
PACKED:    "Hi [name], your GlowCart order is packed and ready for shipping!"
SHIPPED:   "Hi [name], your order is on its way! Expected delivery: 1-3 days."
DELIVERED: "Hi [name], your GlowCart order has been delivered! Enjoy your products ✨"
CANCELLED: "Hi [name], your order #[id] has been cancelled. Contact us for support."
```

Real-Time Architecture (Pusher)

```
Admin updates order status
         ↓
PATCH /api/admin/orders/[id]
         ↓
Prisma updates order in DB
         ↓
Pusher.trigger('order-[trackingCode]', 'status-updated', { status, updatedAt })
         ↓
Customer browser subscribed to same channel receives event
         ↓
React state updates → timeline re-renders instantly
```

AI Prompt for Phase 5

> You are a senior full-stack developer. Build Phase 5 of **GlowCart — real-time order tracking and automated notifications**.
>
> **Context:** Next.js 14, Prisma + PostgreSQL, NextAuth.js, Tailwind CSS, Framer Motion. The Order model has fields: id, trackingCode (unique cuid), status (enum: PLACED/CONFIRMED/PACKED/SHIPPED/DELIVERED/CANCELLED), userId, items, total, shippingAddress, city, region, phone, createdAt.
>
> **What to build:**
>
> 1. **Pusher Setup:** Install `pusher` (server) and `pusher-js` (client). Configure Pusher credentials in .env. Create a Pusher singleton in /lib/pusher.ts (server) and /lib/pusher-client.ts (client).
> 2. **Order Tracking Page (/track/[trackingCode]):** A public page (no auth required). Display a 5-step horizontal progress timeline using Framer Motion: Order Placed → Confirmed → Packed → Shipped → Delivered. Active step pulses with rose color. Completed steps are green with a checkmark. Future steps are gray. Below the timeline, display: estimated delivery note based on status, ordered items list with images, delivery address. Use `pusher-js` to subscribe to the Pusher channel named `order-${trackingCode}`. Listen for the event `status-updated`. On receiving the event, update the order status in React state and re-animate the timeline. Add a "Copy tracking link" button.
> 3. **Update the Admin Order PATCH endpoint** (/api/admin/orders/[id]): After updating the order status in Prisma, trigger a Pusher event: `pusher.trigger('order-' + trackingCode, 'status-updated', { status, updatedAt })`. Then send SMS using Arkesel API (POST to https://sms.arkesel.com/api/v2/sms/send with API key from env). Use the SMS template from the PRD matching the new status. Then send email using Nodemailer with Gmail SMTP — use a styled HTML email template showing order status update, items, and a link to the tracking page.
> 4. **User Dashboard — Order History (/dashboard/orders):** Protected page for logged-in customers. Show all their orders in a list. Each order card shows: order number (truncated), date, status badge (color-coded), total ₵, items count, and a "Track Order" button linking to /track/[trackingCode]. If order is DELIVERED, show a "Leave a Review" link (stub for now).

5. **Nodemailer Email Templates:** Create HTML email templates for each order status. Template must include: GlowCart logo/name, personalized greeting, status update message, order summary table, tracking link, contact info. Make the template responsive (inline CSS, no external stylesheets).
6. Add .env variables for: PUSHER_APP_ID, PUSHER_KEY, PUSHER_SECRET, PUSHER_CLUSTER, ARKESEL_API_KEY, GMAIL_USER, GMAIL_APP_PASSWORD.
7. Add error handling: if SMS or email fails, log the error but do not fail the order status update response. Use try/catch around notification calls.
8. **Notification preference:** Add a `notificationsEnabled` boolean field to the User model. In the /profile page, allow users to toggle SMS/email notifications on or off. Check this flag before sending notifications.

PHASE 6 — TESTING, OPTIMIZATION & DEPLOYMENT

Production-Ready Launch

Objective

Harden the application for production: write tests, optimize performance, fix security gaps, configure environment variables, and deploy to Vercel + Railway with a custom domain.

Features

Testing

- Unit tests (Vitest): utility functions (formatGHS, slugify, calculateDelivery)
- Integration tests (Vitest + Testing Library): product filter UI, add-to-cart flow
- API route tests: Paystack verification, order creation logic
- E2E tests (Playwright): full checkout flow, admin product CRUD, POS scan flow

Performance Optimizations

- Next.js Image component with Cloudinary loader (auto WebP conversion, lazy loading)
- Implement React.lazy + Suspense for Three.js canvas (prevents it from blocking initial load)
- Route-based code splitting (already in Next.js App Router, verify bundle sizes)
- Redis caching for GET /api/products (cache 5 minutes, invalidate on admin product save)
- Debounce search input (300ms)
- Add `loading="lazy"` and proper `sizes` attribute to all images
- Lighthouse target: Performance ≥ 85, SEO ≥ 90, Accessibility ≥ 85

Security

- Rate limiting on auth routes: max 5 login attempts per IP per 15 minutes (using Redis + custom middleware)
- CSRF protection (NextAuth.js handles this)
- Input sanitization on all POST bodies (zod schema validation)

- Helmet-equivalent headers via next.config.js (X-Frame-Options, X-XSS-Protection, CSP)
- Paystack webhook verification (validate webhook signature)
- Admin route middleware: double-check role in every API route, not just UI

SEO

- Next.js Metadata API: unique title + description per page
- OG image for homepage (auto-generated using next/og)
- Sitemap.xml and robots.txt (next-sitemap package)
- Structured data (JSON-LD) on product pages for Google Shopping
- Canonical URLs

PWA

- next-pwa package configuration
- Manifest.json (app name, icons, theme color #C2185B)
- Service worker for offline product browse (cache product pages)
- "Add to Home Screen" prompt on mobile

Deployment

- Vercel: connect GitHub repo, set environment variables, enable automatic deployments on main branch
- Railway: PostgreSQL + Redis instances, connect DATABASE_URL + REDIS_URL to Vercel env
- Cloudflare: point custom domain, enable proxy for DDoS protection + HTTPS
- Environment: separate .env.local (dev) and production env vars in Vercel dashboard

---

AI Prompt for Phase 6

> You are a senior full-stack developer. Build Phase 6 of **GlowCart** — the final production hardening, testing, and deployment phase.
>
> **Context:** Next.js 14 app, Prisma + PostgreSQL (Railway), Redis (Upstash), NextAuth.js, Paystack, Pusher, Cloudinary, Tailwind CSS, Framer Motion, React Three Fiber. The app has 5 completed phases: shopping (Phase 1), admin panel (Phase 2), 3D design (Phase 3), QR POS scanner (Phase 4), and real-time tracking (Phase 5).
>
> **What to build:**
>
> 1. **Zod Validation:** Add Zod schemas for all POST/PATCH API route bodies. Create a /lib/validations/ folder with schemas for: createOrder, createProduct, updateOrder, createScanSession. In each API route, validate req.body against the schema and return 400 with error details on failure.
> 2. **Rate Limiting:** Create a middleware function using Upstash Redis that rate-limits: POST /api/auth/[...nextauth] (max 5/15min per IP), POST /api/orders (max 10/hour per user), POST /api/pos/sessions (max 100/hour per cashier). Return 429 Too Many Requests if exceeded.
> 3. **Security Headers:** In next.config.js, add headers for all routes: X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Referrer-Policy: strict-origin-when-cross-origin, Permissions-Policy: camera=self (camera needed for POS). Add a Content Security Policy that allows: Cloudinary for images, Paystack for scripts, Pusher for WebSockets, Google Fonts for fonts.
> 4. **Redis Caching:** In GET /api/products, check Redis cache first (key: `products:${queryHash}`). If cache hit, return cached data. If miss, query DB, store in Redis with 5-minute TTL, return result. In POST/PUT/DELETE /api/admin/products,

invalidate all `products:*` cache keys.

5. **PWA Configuration:** Install `next-pwa`. Create public/manifest.json with: name "GlowCart", short_name "GlowCart", theme_color "#C2185B", background_color "#FFF8F0", display "standalone", icons at 192x192 and 512x512. Configure next.config.js to enable PWA in production only (disable in development to avoid caching issues).
6. **SEO:** Use Next.js Metadata API to add unique title (e.g., "{Product Name} | GlowCart") and description to /products/[slug] pages. Add JSON-LD structured data (Product schema) to product pages with: name, image, description, price, currency "GHS", availability. Generate sitemap.xml using `next-sitemap` that includes all product and category pages.
7. **Playwright E2E Tests:** Write 3 E2E test specs: (a) checkout-flow.spec.ts — visit homepage, click a product, add to cart, go to checkout, fill form, mock Paystack success, verify order confirmed page. (b) admin-product.spec.ts — log in as admin, go to /admin/products, click "Add Product", fill form, save, verify product appears in list. (c) pos-scanner.spec.ts — log in as cashier, go to /pos, mock a QR scan result by calling the scan handler directly with a product ID, verify item appears in session list, verify total updates.
8. **Paystack Webhook:** Create POST /api/webhooks/paystack route. Validate the `x-paystack-signature` header (HMAC SHA-512 of request body using your Paystack secret key). On valid `charge.success` event: find the order by payment reference, update paymentStatus to "paid", and if status is still PLACED set it to CONFIRMED. Return 200.
9. **Environment Variables Audit:** Create a comprehensive .env.example file listing every environment variable used across all phases with descriptions. Group them by service: Database, Auth, Paystack, Cloudinary, Pusher, Arkesel, Gmail, Redis.
10. **Deployment Checklist:** Create a DEPLOYMENT.md file in the repo root documenting: how to set up Railway PostgreSQL and run `prisma db push`, how to configure Vercel project with env variables, how to connect a custom .com.gh domain on Cloudflare with SSL, how to test the Paystack webhook using their dashboard, and how to seed the production database with initial products.

PHASE 7 — ADD-ONS

AI Recommender · Review System · Subscription Boxes · Abandoned Cart Recovery · Loyalty Points

---

Objective

Layer five high-impact features on top of the completed core platform. Each add-on is independent — they can be built in any order. Together they significantly increase customer retention, average order value, and recurring revenue.

ADD-ON 1: AI PRODUCT RECOMMENDER

"You May Also Like" — Powered by Claude API

Features

Where Recommendations Appear

- Product detail page: "You May Also Like" section (4 cards) below the main product
- Cart drawer: "Pair It With" section showing 2 complementary items

- Order confirmed page: “Complete Your Routine” section (3 suggestions)
- Homepage: “Picked For You” section for logged-in users (based on order history)

How It Works

- Logged-in user with order history: call Claude API with user’s last 5 purchased product names + current product being viewed → Claude returns 3–4 product names that complement them → look up those products in DB by name similarity
- Guest user / no history: use category-based rules (viewing Body Spray → suggest Deodorant + Skincare)
- Fallback: if Claude API call fails, fall back to most popular products in same category
- Results are cached in Redis per user (key: `recs:userId:productId`, TTL: 30 minutes)

Claude API Prompt Used (server-side)

```
System: You are a beauty product recommender for a Ghanaian cosmetics store called GlowCart.
Given the products a customer has bought or is viewing, suggest complementary products from
the available catalog. Return ONLY a JSON array of product names, no explanation.
Example: ["Rose Body Lotion", "Vitamin C Serum", "Hydrating Toner"]

User: The customer is currently viewing: [CURRENT_PRODUCT].
Their recent purchases: [PRODUCT_1, PRODUCT_2, ...].
Available products in our catalog: [FULL_PRODUCT_NAME_LIST].
Suggest 4 complementary products they are most likely to buy next.
```

Database Changes

No new tables. Add a `viewCount` Int field to the Product model to track popularity for fallback logic.

```
model Product {
  // ... existing fields
  viewCount  Int  @default(0)
}
```

API Endpoints (Add-On 1)

Method	Endpoint	Description
GET	/api/recommendations?productId=X&userId=Y	Get AI recommendations
POST	/api/products/[id]/view	Increment product view count

AI Prompt for Add-On 1

> You are a senior full-stack developer. Add an **AI-powered product recommendation feature** to GlowCart, a Next.js 14 cosmetics e-commerce app using Prisma + PostgreSQL, Redis (Upstash), and the Anthropic Claude API.
>
> **What to build:**
>
> 1. Create GET /api/recommendations route. Accept query params: `productId` (current product) and `userId` (optional). Logic: (a) Fetch current product name from DB. (b) If userId provided, fetch their last 5 purchased product names from OrderItems. (c) Fetch all published product names from DB (for the catalog list). (d) Check Redis cache: key `recs:${userId ?? 'guest'}:${productId}`, TTL 30 minutes. If cache hit, return cached array. (e) If cache miss, call the Anthropic Claude API (model: claude-sonnet-4-20250514) with the prompt described in the PRD. Parse the JSON array

response. (f) Look up each returned product name in the DB using a case-insensitive `contains` search. (g) Return matching Product objects (id, name, price, images[0], slug). (h) Cache the result in Redis for 30 minutes.

2. Add a `viewCount Int @default(0)` field to the Product model in Prisma schema. Create POST /api/products/[id]/view that increments the view count. Call this from the product detail page on load (fire-and-forget, don't await in the component).
3. Build a reusable `<RecommendationGrid>` React component. Props: `productId`, `userId` (optional), `title` (string). Fetches from /api/recommendations on mount. Shows a loading skeleton (4 product card skeletons) while fetching. Renders 4 product cards in a horizontal scroll row on mobile, 4-column grid on desktop. Each card is the standard ProductCard component.
4. Add `<RecommendationGrid title="You May Also Like" productId={product.id} userId={session?.user?.id} />` to the product detail page, below the product info.
5. In the cart drawer, show a `<RecommendationGrid title="Pair It With" ... />` showing 2 cards (pass `limit=2` param to the API) when cart has at least 1 item. Use the first item in the cart as the `productId`.
6. On the order confirmed page, show `<RecommendationGrid title="Complete Your Routine" ... />` with 3 cards based on the first item in the completed order.
7. For logged-in users on the homepage, show a `<RecommendationGrid title="Picked For You" userId={session.user.id} productId={null} />`. When productId is null, the API should use only the user's order history (last 3 purchases) as the context.
8. Fallback behavior: if the Claude API call fails or returns unparseable JSON, fall back to fetching the top 4 most-viewed products in the same category as the current product (ordered by viewCount desc).
9. Store your Anthropic API key in .env as `ANTHROPIC_API_KEY`. Never expose it on the client side — all Claude API calls must be server-side only in API routes.

ADD-ON 2: REVIEW SYSTEM WITH PHOTO UPLOAD

Customer Reviews, Ratings & Photo Evidence

Features

Customer Review Submission

- Review form appears on product detail page only for customers who have purchased and received (DELIVERED status) that product — enforced server-side
- Form fields: star rating (1–5, interactive star selector), written review (min 20 chars, max 500 chars), optional photo upload (up to 2 photos via Cloudinary, max 5MB each)
- After submission: review goes into "pending" state (not shown publicly until admin approves)
- Customer can edit or delete their own review

Review Display on Product Page

- Average star rating shown prominently near product name (e.g., ★★★★☆ 4.2 | 38 reviews)
- Rating breakdown bar chart: how many gave 5★, 4★, 3★, 2★, 1★
- List of approved reviews: reviewer name (first name + last initial), date, star rating, review text, photo thumbnails (click to enlarge)

- “Most Helpful” sort (upvotes) and “Most Recent” sort
- Helpful/Not Helpful vote buttons per review
- Pagination: 5 reviews per page

Admin Review Moderation (/admin/reviews)

- Table showing all pending reviews: product name, reviewer, rating, excerpt, date, Approve / Reject buttons
- View full review before deciding
- Filter by: pending, approved, rejected
- Bulk approve option

Database Tables (Add-On 2)

```
model Review {
  id          String        @id @default(cuid())
  userId      String
  productId   String
  orderId     String
  rating      Int           // 1-5
  body        String
  photos      String[]      // Cloudinary URLs
  status      ReviewStatus  @default(PENDING)
  helpfulVotes Int          @default(0)
  createdAt   DateTime      @default(now())
  user        User          @relation(fields: [userId], references: [id])
  product     Product       @relation(fields: [productId], references: [id])
  votes       ReviewVote[]
}

model ReviewVote {
  id        String  @id @default(cuid())
  reviewId  String
  userId    String
  isHelpful Boolean
  review    Review  @relation(fields: [reviewId], references: [id])
  user      User    @relation(fields: [userId], references: [id])
  @@unique([reviewId, userId])
}

enum ReviewStatus {
  PENDING
  APPROVED
  REJECTED
}
```

API Endpoints (Add-On 2)

Method	Endpoint	Description
GET	/api/products/[id]/reviews	Get approved reviews for product
POST	/api/products/[id]/reviews	Submit a review (auth required)
PUT	/api/reviews/[id]	Edit own review

DELETE	/api/reviews/[id]	Delete own review
POST	/api/reviews/[id]/vote	Vote helpful/not helpful
GET	/api/admin/reviews	Get all reviews (filtered)
PATCH	/api/admin/reviews/[id]	Approve or reject review

AI Prompt for Add-On 2

> You are a senior full-stack developer. Add a **customer review system with photo upload** to GlowCart, a Next.js 14 app with Prisma + PostgreSQL, NextAuth.js, Cloudinary, and Tailwind CSS.
>
> **What to build:**
>
> 1. Add these Prisma models: `Review` (id, userId, productId, orderId, rating [Int 1–5], body [String], photos [String[]], status [enum: PENDING/APPROVED/REJECTED, default PENDING], helpfulVotes [Int default 0], createdAt) and `ReviewVote` (id, reviewId, userId, isHelpful [Boolean], with unique constraint on [reviewId, userId]).
> 2. Build the GET /api/products/[id]/reviews endpoint: return only APPROVED reviews for the product, with user first name + last initial (never full name), paginated at 5 per page. Include aggregate data: average rating, count per star level (1–5). Accept query params: `sort` (recent or helpful), `page` .
> 3. Build POST /api/products/[id]/reviews: auth required. Validate that the requesting user has an Order with status DELIVERED that contains this productId (check OrderItems). If not, return 403 "You can only review products you have purchased and received." Validate: rating is 1–5, body is 20–500 chars, max 2 photo URLs. Save review with status PENDING.
> 4. On the product detail page, above the "You May Also Like" section, add: (a) A `<RatingSummary>` component showing the average star rating prominently with a star icon row and the review count. Below it, a horizontal bar chart showing the distribution (5★: 40%, 4★: 30%, etc.) styled with Tailwind. (b) A `<ReviewList>` component that fetches and paginates reviews. Each review card shows: stars, reviewer name, date, review body, photo thumbnails (if any). Photos open in a lightbox on click. (c) Below the review list, if the logged-in user has purchased and received this product and has not yet reviewed it, show a `<ReviewForm>` with: an interactive 5-star selector (clicking star fills stars 1 through N), textarea, and drag-and-drop photo uploader (uploads to Cloudinary via /api/admin/upload endpoint, returns URL). On submit, call POST /api/products/[id]/reviews and show a "Review submitted for approval" success message.
> 5. Build POST /api/reviews/[id]/vote: auth required, one vote per user per review (upsert ReviewVote). Update the helpfulVotes count on the Review record.
> 6. Build the Admin Reviews page (/admin/reviews): a table showing all PENDING reviews first. Columns: product thumbnail, product name, reviewer first name, star rating, review excerpt (50 chars), date, Approve button (green), Reject button (red). On approve, set status to APPROVED. On reject, set status to REJECTED. Include a filter tab bar: All / Pending / Approved / Rejected.
> 7. Protect POST/PUT/DELETE review endpoints: users can only edit or delete their own reviews (check userId === session.user.id).
> 8. Add `reviews Review[]` relation to both User and Product models in Prisma schema.

ADD-ON 3: SUBSCRIPTION BOXES

Monthly Cosmetics Bundles via Paystack Recurring Billing

Features

Subscription Plans (Defined by Admin)

- Admin creates subscription plans: name, description, list of included products or “mystery box”, price per month (GHS), billing cycle (monthly), image
- Example plans: “Glow Starter Box” (₵85/month), “Premium Skin Bundle” (₵150/month), “Full Routine Box” (₵220/month)

Customer Subscription Flow

- /subscriptions page: shows all active plans with descriptions, included items, and price
- “Subscribe Now” button → checkout with Paystack recurring billing
- Paystack creates a recurring plan using their Plans + Subscriptions API
- Customer is billed automatically every 30 days
- Customer receives confirmation email on each successful billing

Customer Subscription Management (/dashboard/subscriptions)

- View active subscriptions with next billing date
- Pause subscription (up to 2 months)
- Cancel subscription (takes effect at end of billing period)
- View subscription order history (each month’s box delivery)

Admin Subscription Management (/admin/subscriptions)

- View all active subscribers
- See monthly recurring revenue (MRR) on analytics dashboard
- Manage plans (create, edit, deactivate)
- See churn rate: cancelled subscriptions this month

Paystack Recurring Billing Flow

```
1. Create a Plan on Paystack (one-time setup per plan type)
   POST https://api.paystack.co/plan
   { name, amount (in pesewas), interval: "monthly" }

2. Customer subscribes → initialize transaction with plan code
   POST https://api.paystack.co/transaction/initialize
   { email, amount, plan: "PLN_xxxxxxxx" }

3. Paystack handles recurring charges automatically

4. Paystack sends webhook on each charge.success event
   → Create a SubscriptionOrder in DB
   → Send confirmation email to customer
```

Database Tables (Add-On 3)

```
model SubscriptionPlan {
  id            String   @id @default(cuid())
  name          String
  description   String
  price         Float    // GHS per month
```

```
  paystackPlanCode String @unique
  image         String?
  isActive      Boolean  @default(true)
  features      String[] // list of included product descriptions
  createdAt     DateTime @default(now())
  subscriptions UserSubscription[]
}

model UserSubscription {
  id                String           @id @default(cuid())
  userId            String
  planId            String
  paystackSubCode   String           @unique // Paystack subscription code
  status            SubStatus        @default(ACTIVE)
  nextBillingDate   DateTime
  startedAt         DateTime         @default(now())
  cancelledAt       DateTime?
  user              User             @relation(fields: [userId], references: [id])
  plan              SubscriptionPlan @relation(fields: [planId], references: [id])
  orders            SubscriptionOrder[]
}

model SubscriptionOrder {
  id             String           @id @default(cuid())
  subscriptionId String
  amount         Float
  paymentRef     String
  deliveryStatus OrderStatus      @default(PLACED)
  createdAt      DateTime         @default(now())
  subscription   UserSubscription @relation(fields: [subscriptionId], references: [id])
}

enum SubStatus {
  ACTIVE
  PAUSED
  CANCELLED
  EXPIRED
}
```

AI Prompt for Add-On 3

> You are a senior full-stack developer. Add a **monthly subscription box feature** to GlowCart using Paystack's recurring billing API. The app uses Next.js 14, Prisma + PostgreSQL, NextAuth.js, Tailwind CSS, and Paystack for all payments (Ghana market).
>
> **What to build:**
>
> 1. Add Prisma models: `SubscriptionPlan` (id, name, description, price, paystackPlanCode, image, isActive, features [String[]], createdAt), `UserSubscription` (id, userId, planId, paystackSubCode, status [enum: ACTIVE/PAUSED/CANCELLED/EXPIRED], nextBillingDate, startedAt, cancelledAt), `SubscriptionOrder` (id, subscriptionId, amount, paymentRef, deliveryStatus, createdAt).
> 2. Build /subscriptions page: fetch all active SubscriptionPlans from DB and display them as plan cards. Each card shows: plan image, name, price (₵/month), features list (bullet points), and a "Subscribe Now" button. Style with a gradient border on hover using Tailwind.
> 3. Build POST /api/subscriptions/checkout: auth required. Accept planId. (a) Fetch the plan from DB to get paystackPlanCode and price. (b) Call Paystack's initialize transaction endpoint (POST https://api.paystack.co/transaction/initialize) with: email from session, amount in pesewas (price * 100), plan: paystackPlanCode. (c) Return the Paystack authorization URL. (d) Redirect customer to Paystack's hosted page where

> they enter card/mobile money details and authorize recurring billing.
>
> 4. Update POST /api/webhooks/paystack to handle subscription events: on `subscription.create` event, create a new UserSubscription record using the subscription_code and next payment date from the webhook payload. On `invoice.payment_failed`, update status to CANCELLED. On `charge.success` with a subscription_code in the payload, create a new SubscriptionOrder record and send a confirmation email.
> 5. Build /dashboard/subscriptions page (auth required): show all of the logged-in user's subscriptions. Each subscription card shows: plan name, status badge, next billing date, and two buttons — "Pause" (calls Paystack's disable subscription endpoint then updates DB) and "Cancel" (calls Paystack's disable subscription endpoint, sets cancelledAt in DB). Below subscriptions, list all SubscriptionOrders (past boxes) with delivery status.
> 6. Build the Admin Subscriptions section (/admin/subscriptions): (a) A "Plans" tab: table of all plans with Edit/Deactivate buttons. An "Add Plan" button opens a form to create a plan — this form calls POST https://api.paystack.co/plan to create the plan on Paystack first, then stores the returned plan_code in the DB. (b) A "Subscribers" tab: table of all UserSubscriptions showing customer name, plan, status, start date, next billing date. (c) Add a Monthly Recurring Revenue (MRR) stat card to the main analytics dashboard: sum of (active subscriptions × plan price).
> 7. Add the Paystack secret key header `Authorization: Bearer ${process.env.PAYSTACK_SECRET_KEY}` to all Paystack API calls server-side.

ADD-ON 4: ABANDONED CART RECOVERY

Automated Email Re-engagement 1 Hour After Cart Abandonment

Features

How It Works

- When a logged-in user adds items to their cart but does NOT complete checkout within 60 minutes, a recovery email is automatically sent
- Email shows the abandoned items with images, names, prices, and a direct "Complete Your Purchase" CTA button linking back to /checkout with their cart pre-loaded
- If user completes purchase after receiving email, the abandonment is resolved and no further emails are sent
- Only one email per abandonment event (no spam)
- Admin can see abandoned cart statistics on the dashboard

Trigger Mechanism (using Vercel Cron Jobs)

- A cron job runs every 15 minutes: POST /api/cron/abandoned-carts
- It queries for CartItems where the user has not placed an order in the last 60–90 minutes
- Checks `lastEmailSentAt` on the cart — if null and cart is older than 60 minutes, send email and update flag
- Cron is secured with a `CRON_SECRET` header check

Recovery Email Content

- Subject: "You left something behind ✨ | GlowCart"
- Body: personalized greeting, list of abandoned items (image, name, price), total value, a bold "Complete Purchase" button, and optionally a small discount code (configurable)

Database Changes (Add-On 4)

```
// Add to CartItem model or create a Cart model to track metadata
model Cart {
  id                String    @id @default(cuid())
  userId            String    @unique
  user              User      @relation(fields: [userId], references: [id])
  lastActivityAt    DateTime  @default(now()) @updatedAt
  recoveryEmailSentAt DateTime?
  items             CartItem[]
}

// Update CartItem to link to Cart instead of directly to User
model CartItem {
  id        String  @id @default(cuid())
  cartId    String
  productId String
  quantity  Int
  cart      Cart    @relation(fields: [cartId], references: [id])
  product   Product @relation(fields: [productId], references: [id])
}
```

API Endpoints (Add-On 4)

Method	Endpoint	Description
POST	/api/cron/abandoned-carts	Cron: find and email abandoned carts
GET	/api/admin/stats/abandoned-carts	Admin analytics: abandonment rate

---

AI Prompt for Add-On 4

> You are a senior full-stack developer. Add an **abandoned cart recovery** feature to GlowCart. The app uses Next.js 14, Prisma + PostgreSQL, Nodemailer (Gmail SMTP), Vercel Cron Jobs, and Tailwind CSS.
>
> **What to build:**
>
> 1. Refactor the cart data model: create a `Cart` model (id, userId [unique], lastActivityAt [updatedAt auto], recoveryEmailSentAt [DateTime?]). Move CartItems to link to cartId instead of userId. Every time a CartItem is added, updated, or removed, touch the `lastActivityAt` on the Cart (Prisma's `@updatedAt` handles this automatically).
> 2. Update all existing cart API routes (/api/cart) to use the new Cart model — find or create the Cart by userId, then operate on CartItems through the Cart.
> 3. Create POST /api/cron/abandoned-carts: (a) Verify the request has header `Authorization: Bearer ${process.env.CRON_SECRET}`. Return 401 if missing. (b) Query all Carts where: `lastActivityAt` is more than 60 minutes ago AND `recoveryEmailSentAt` IS NULL AND the cart has at least one item. (c) For each cart, fetch the user's email and name, fetch the CartItems with product details (name, price, image). (d) Send an HTML recovery email using Nodemailer. Email subject: "You left something behind ✨ | GlowCart". Include: user's first name, list of abandoned items (product image, name, price in ₵), cart total, and a CTA button "Complete Your Purchase" linking to https://glowcart.com/checkout. (e) Update `recoveryEmailSentAt` to now() on the Cart record. (f) Return a JSON summary: { processed: N, emailsSent: N }.
> 4. Configure Vercel Cron in vercel.json:
>
>    ```
>    {
>    ```

```
  "crons": [{ "path": "/api/cron/abandoned-carts", "schedule": "*/15 * * * *" }]
}
```

Add `CRON_SECRET` to .env for securing the endpoint.

5. Reset `recoveryEmailSentAt` to null and update `lastActivityAt` when a user adds a new item to their cart after the recovery email was sent (so if they abandon again later, a new email will trigger).
6. When an order is successfully placed (POST /api/orders), delete all CartItems and reset `recoveryEmailSentAt = null` on the user's Cart so the abandonment flag is cleared.
7. Create a responsive HTML email template (inline CSS only — no external stylesheets, as email clients strip them). The template must include: GlowCart brand name at top, greeting, items table (product name, quantity, price), total row, the CTA button styled with rose background (#C2185B), and a footer with "You received this because you have items in your cart" and an unsubscribe link.
8. Add an "Abandoned Carts" stat to the admin dashboard: count of carts older than 60 minutes with items and no completed order in the last 24 hours. Show as a stat card with a warning color.

ADD-ON 5: LOYALTY POINTS SYSTEM

Earn Points Per Purchase · Redeem for Discounts

Features

Earning Points

- Customers earn 1 point per ₵1 spent on completed orders (configurable rate)
- Bonus points for: first order (+50 points), writing a review (+20 points), referring a friend (+100 points each)
- Points are credited only when order status reaches DELIVERED
- Points never expire (or configurable expiry of 12 months)

Redeeming Points

- On checkout, customers see their current points balance
- They can choose to redeem points for a discount: 100 points = ₵1 off (configurable)
- Minimum redemption: 500 points (₵5 off). Maximum: 50% of order total
- On redemption: points are deducted immediately on order placement
- If order is cancelled, points are refunded

Customer Points Dashboard (/dashboard/points)

- Current points balance with animated counter
- Points history: earned and spent transactions with dates and descriptions
- Progress bar to next reward tier (if tiered loyalty is enabled)
- How to earn more points (tips section)

Loyalty Tiers (Optional Gamification)

- Bronze: 0–499 lifetime points

- Silver: 500–1999 lifetime points (5% bonus on every purchase)
- Gold: 2000+ lifetime points (10% bonus + free delivery on all orders)
- Tier badge shown on user profile and order confirmation

Admin Loyalty Management (/admin/loyalty)

- Configure points earn rate (points per ₵1)
- Configure redemption rate (points per ₵1 discount)
- View total outstanding points liability
- Manually award or deduct points for a customer (with reason)

Database Tables (Add-On 5)

```
model LoyaltyAccount {
  id              String          @id @default(cuid())
  userId          String          @unique
  pointsBalance   Int             @default(0)
  lifetimePoints  Int             @default(0)  // never decreases
  tier            LoyaltyTier     @default(BRONZE)
  user            User            @relation(fields: [userId], references: [id])
  transactions    PointTransaction[]
}

model PointTransaction {
  id          String            @id @default(cuid())
  accountId   String
  points      Int               // positive = earned, negative = spent
  type        TransactionType
  description String
  orderId     String?
  createdAt   DateTime          @default(now())
  account     LoyaltyAccount    @relation(fields: [accountId], references: [id])
}

enum LoyaltyTier {
  BRONZE
  SILVER
  GOLD
}

enum TransactionType {
  PURCHASE_EARN
  REVIEW_BONUS
  REFERRAL_BONUS
  FIRST_ORDER_BONUS
  ADMIN_ADJUSTMENT
  REDEMPTION
  REFUND
}
```

API Endpoints (Add-On 5)

Method	Endpoint	Description

GET	/api/loyalty	Get current user's points balance + tier
GET	/api/loyalty/history	Get user's points transaction history
POST	/api/loyalty/redeem	Apply points redemption to checkout
POST	/api/admin/loyalty/adjust	Admin manually adjust user points
GET	/api/admin/loyalty/stats	Total outstanding points liability

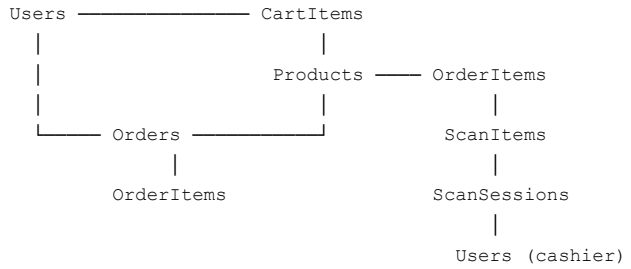
---

AI Prompt for Add-On 5

> You are a senior full-stack developer. Add a **loyalty points and rewards system** to GlowCart. The app uses Next.js 14, Prisma + PostgreSQL, NextAuth.js, Tailwind CSS, and Framer Motion.
>
> **What to build:**
>
> 1. Add Prisma models: `LoyaltyAccount` (id, userId [unique], pointsBalance [Int default 0], lifetimePoints [Int default 0], tier [enum: BRONZE/SILVER/GOLD default BRONZE]) and `PointTransaction` (id, accountId, points [Int — positive=earned, negative=spent], type [enum: PURCHASE_EARN/REVIEW_BONUS/REFERRAL_BONUS/FIRST_ORDER_BONUS/ADMIN_ADJUSTMENT/REDEMPTION/REFUND], description [String], orderId [optional], createdAt).
> 2. Create a LoyaltyAccount automatically for every new user on registration (hook into the NextAuth `createUser` callback).
> 3. **Earning points on delivery:** In the order status update logic (PATCH /api/admin/orders/[id]), when status changes to DELIVERED: (a) Calculate earned points = Math.floor(order.total) × 1 (1 point per ₵1). (b) Apply tier multiplier: SILVER = 1.05x, GOLD = 1.10x. (c) Update LoyaltyAccount: increment pointsBalance and lifetimePoints. (d) Create a PointTransaction (type: PURCHASE_EARN, description: "Order #[id] delivered"). (e) Recalculate tier: lifetimePoints >= 2000 → GOLD, >= 500 → SILVER, else BRONZE.
> 4. **First order bonus:** In POST /api/orders, after creating the order, check if this is the user's first ever order. If yes, create a PointTransaction of +50 points (type: FIRST_ORDER_BONUS) and update pointsBalance.
> 5. **Points redemption at checkout:** On the checkout page, below the order summary, show: "Your points balance: [N] points (worth ₵[N/100])". A "Use Points" toggle. If toggled on and user has ≥ 500 points, show a slider to select how many points to redeem (max: min(pointsBalance, Math.floor(order.total * 100 / 2)) — capped at 50% of total). Update the order total in real-time. On POST /api/loyalty/redeem: validate the user has enough points, deduct from balance, create PointTransaction (type: REDEMPTION), return the discount amount (Float, GHS). Store `pointsRedeemed` and `pointsDiscount` on the Order model (add these fields to Prisma).
> 6. **Points Dashboard (/dashboard/points):** Show: (a) A large animated counter of current points balance using Framer Motion's `useSpring` + `useTransform` for count-up animation on load. (b) Tier badge (bronze/silver/gold with matching icon and color). (c) Progress bar to next tier: show points needed to reach next tier. (d) Points transaction history table: date, description, points change (+/-), running balance. (e) "How to Earn More" section: list of all earning opportunities (purchase, review, refer a friend).
> 7. **Admin Loyalty page (/admin/loyalty):** (a) Stat card: Total outstanding points liability = sum of all LoyaltyAccount pointsBalance values, converted to ₵ equivalent. (b) "Adjust Points" form: search customer by name/email, enter points amount (positive or negative), enter reason, submit calls POST /api/admin/loyalty/adjust which creates a PointTransaction (type: ADMIN_ADJUSTMENT) and updates balance. (c) Top 10 most loyal customers by lifetimePoints.
> 8. **Review bonus integration:** In POST /api/products/[id]/reviews (from Add-On 2), after saving the review, also award +20 loyalty points to the reviewer (PointTransaction type: REVIEW_BONUS).
> 9. Display the user's tier badge and point balance in the site navbar (next to their avatar) when logged in.

---

11. FULL DATABASE SCHEMA SUMMARY



All monetary values stored as `Float` in GHS. All IDs use `cuid()` for security (no sequential IDs exposed publicly). Soft deletes: use `isPublished: false` instead of hard deletes on products.

---

13. GHANA-SPECIFIC CONSIDERATIONS

Concern	Solution
Payment	Paystack supports MTN MoMo, Vodafone Cash, AirtelTigo, Visa/MC in Ghana
Currency	All prices in GHS (₵). Format: `₵ 45.00`
SMS Provider	Arkesel (Ghanaian provider) or Africa's Talking
Phone format	Validate Ghana numbers: +233XX XXX XXXX (10 digits starting with 0)
Regions	All 16 Ghana regions in dropdown (Greater Accra, Ashanti, Western, etc.)
Internet	Optimize for low bandwidth: compress images, lazy load, minimal JS bundles
Mobile	80%+ of Ghanaian users shop on mobile — mobile-first design is critical
VAT	Standard rate is 21.9% (VAT + NHIL + GetFund) — confirm with brand if applicable
Delivery	Show delivery areas clearly (Accra, Kumasi, Takoradi etc.) with separate fees
Language	English only (primary language in Ghana)

Document Version: 2.0 | Prepared for: GlowCart Single-Brand Cosmetics Platform Stack: Next.js 14 · PostgreSQL · Prisma · Paystack · Cloudinary · Pusher · Arkesel · Claude API